

Examen Horas de Vuelo

▼ Instrucciones de examen

1. Un solo chat
2. 1:30hr para completar
3. Comentarios, regulaciones de programacion, estructura logica
4. Sustentar que se esta consultando
5. Todo lo mas profesional porque se compartira el chat
6. Archivo

[Prueba_Diagnostica_Campuslands_Guatemala_Postulaciones.pdf](#)



Entendimiento de Proyecto - PRUEBA DIAGNÓSTICA DE DESARROLLO BACKEND

1. Metodo de evaluacion: logica aplicada, la calidad de las decisiones técnicas, la organización del proyecto, el manejo de errores y la claridad de la solución.
2. Tecnologias: NodeJs, Librerias if you must, Bases de datos relacional basada en SQL.
3. Caso de Estudio:
 - a. Empresa requiere un API REST
 - b. Gestionar las postulaciones de candidatos a sus vacantes laborales.
 - c. Proceso de usuario:
 - i. Empresa publica la vacante
 - ii. Candidato consulta informacion de la vacante

- iii. Candidato se postula
- iv. Informacion se guarda en una bd
- v. Equipo de seleccion revisa y consulta las postulaciones
 - 1. Necesidad:
 - a. Segùn prioridad de revision: refiere a las reglas de negocio
 - b. Efecto: primero los perfiles màs afines a la vacante
- d. Informacion de BD
 - i. Candidato
 - 1. Id candidato
 - 2. Nombre
 - 3. correo
 - 4. años de experiencia
 - ii. Vacante
 - 1. id vacanet
 - 2. titulo de cargo
 - 3. años minimos de experiencia
 - 4. estado de vacante
 - iii. Postulacion
 - 1. id postulacion
 - 2. id candidato
 - 3. id vacante
 - 4. carta de presentacion
 - 5. fuente de postulacion
 - 6. puntaje
 - 7. prioridad
 - 8. estado

9. fecha de creacion

10. fecha de ultima actualizacion de estado

e. Requerimientos de diseño:

i. Diseñar las tablas

ii. Tipos de datos

iii. primary keys

iv. foreign keys

v. restricciones

vi. demas elementos necesarios

vii. explicar relaciones entre candidatos - vacantes - postulaciones

viii. No se exige implementar endpoints para crear candidatos y vacantes

ix. seed de al menos 3 candidatos y 2 vacantes (1 en estado CLOSED)

f. Valores Permitidos:

i. Fuentes de postulacion

1. REFERRAL

2. INTERNAL

3. JOB_BOARD

4. OTHER

ii. Estados de postulacion

1. RECIEVED

2. IN_REVIEW

3. REJECTED

4. HIRED

iii. Estado de Vacante

1. OPEN

2. CLOSED

3. Tomar en cuenta: se consideran postulaciones activas aquellas en estado RECIEVED o IN_REVIEW

4. Estados finales: REJECTED y HIRED

g. Calculo de Prioridad

i. Requerimiento: el calculo de prioridad se realiza en el backend, no es valor proporcionado por usuario.

ii. Condicion

1. +4

Los años de experiencia del candidato son iguales o superiores a los mínimos exigidos por la vacante

2. +3

La fuente es REFERRAL

3. +2

La fuente es INTERNAL

4. +2

La carta de presentación contiene alguna de las palabras "node", "sql" o "api"

5. +1

La carta de presentación tiene más de 500 caracteres

6. -2

El candidato tiene 3 o más postulaciones activas en otras vacantes

iii. Prioridad

1. TOP puntaje 7+

2. HIGH puntaje 5 - 6

3. MEDIUM puntaje 3 - 4

4. LOW puntaje 0 - 2

h. Funciones Obligatorias

i. Registrar una postulación

Implementar:

POST /applications

Ejemplo de cuerpo de solicitud:

```
{  
  "candidateId": 3,  
  "vacancyId": 1,  
  "source": "REFERRAL",  
  "coverLetter": "I have four years of experience building REST APIs  
with Node.js and SQL databases"  
}
```

La operación debe, como mínimo:

1. Validar los datos obligatorios.
2. Validar que la fuente de la postulación sea permitida.
3. Verificar que el candidato exista.
4. Verificar que la vacante exista y se encuentre en estado OPEN.
5. Aplicar la regla de duplicidad descrita en la sección 8.
6. Consultar la información necesaria para aplicar las reglas de prioridad.
7. Calcular el puntaje y determinar la prioridad.
8. Registrar la postulación en la base de datos.
9. Asignar RECEIVED como estado inicial.
10. Retornar una respuesta HTTP coherente con el resultado de la operación.

ii. Consultar postulaciones

Implementar:

GET /applications

La respuesta debe incluir la información de la postulación y, como

mínimo, el nombre y correo

electrónico del candidato y el título de la vacante.

Los resultados deben retornarse ordenados por puntaje de mayor a menor y, en caso de empate, por

fecha de creación (la más antigua primero).

También debe permitir filtrar por estado, por vacante o por ambos criterios combinados:

GET /applications?status=IN_REVIEW

GET /applications?status=RECEIVED&vacancyId=1

iii. Cambiar el estado de una postulación

Implementar:

PUT /applications/:id/status

Ejemplo de cuerpo de solicitud:

```
{  
  "status": "IN_REVIEW"  
}
```

La operación debe validar:

- Que la postulación exista.
- Que el nuevo estado corresponda a uno de los estados permitidos.
- Que una postulación en estado final (REJECTED o HIRED) no pueda cambiar de estado.
- Que se actualice la fecha de última actualización de estado.
- Que los errores sean respondidos mediante códigos HTTP apropiados.

i. Regla de limitacion

- i. Los candidatos no pueden postularse a las vacantes que ya tienen postulado. Solamente pueden volver a postular cuando hayan transcurrido minimop 30 dias desde la fecha de rechazo.

j. Implementacion de Pruebas

- i. implementar 3 priebas automatizadas relacionadas con la logica de calculo de puntaje y prioridad.

k. Fundamentos de IA

- i. Agregar un archivo de RESPUESTAS.md

I. Entregables

- i. Código fuente de la aplicación.
- ii. package.json y archivo de bloqueo de dependencias generado por el gestor utilizado.
- iii. database.sql con las instrucciones necesarias para crear las tablas y estructuras de base de datos requeridas, junto con los datos de prueba de candidatos y vacantes.
- iv. .env.example con las variables de entorno necesarias, sin contraseñas o credenciales reales.
- v. RESPUESTAS.md con las respuestas de la sección de inteligencia artificial.
- vi. README.md con instrucciones para instalar dependencias, configurar la base de datos, ejecutar la aplicación y probar los endpoints
- vii. Las pruebas automatizadas implementadas.
- viii. No incluya la carpeta node_modules en la entrega
- ix. Se valorará el uso adecuado de variables de entorno y la ausencia de credenciales sensibles en el código.
- x. Se valorará la separación de responsabilidades, legibilidad, nombres adecuados y manejo consistente de errores.
- xi. Las decisiones técnicas que considere relevantes pueden explicarse brevemente en README.md.

▼ Desarrollo de Prompt

Para el desarrollo de este proyecto se estará implementando un diseño de prompt CREO:

- **Contexto (El "Dónde estamos"):**

Buenos días Claude, un gusto saludarte. Necesito diseñar un sistema que responda a las necesidades de una empresa. Se necesita una API REST para gestionar las postulaciones de candidatos a sus vacantes laborales. Los candidatos se postulan a vacantes publicadas por la empresa, y el equipo de selección necesita almacenar las postulaciones, consultar su estado y calcular automáticamente una prioridad de revisión utilizando reglas de negocio, de modo que los reclutadores atiendan primero los perfiles más afines.

- **Rol (El "Quién responde"):**

Necesito que actúes como un API Platform Integration Specialist, Database Administrator (DBA), Query Optimization Specialist o el rol que tu analices conveniente para el desarrollo de este sistema.

- **Entrada / Tarea (El "Qué hacer"):**

Analiza el siguiente entendimiento de proyecto que escribí, crea un brief explicando el proyecto y el desarrollo necesario con sus requerimientos para que pueda entender tu análisis. Puedes hacerme preguntas si es necesario. Entendimiento =

1. Metodo de evaluacion: logica aplicada, la calidad de las decisiones técnicas, la organización del proyecto, el manejo de errores y la claridad de la solución.
2. Tecnologías: NodeJs, Librerías if you must, Bases de datos relacional basada en SQL.
3. Caso de Estudio:
 - a. Empresa requiere un API REST
 - b. Gestionar las postulaciones de candidatos a sus vacantes laborales.
 - c. Proceso de usuario:

- i. Empresa publica la vacante
- ii. Candidato consulta informacion de la vacante
- iii. Candidato se postula
- iv. Informacion se guarda en una bd
- v. Equipo de seleccion revisa y consulta las postulaciones
 - 1. Necesidad:
 - a. Segùn prioridad de revision: refiere a las reglas de negocio
 - b. Efecto: primero los perfiles màs afines a la vacante
- d. Informacion de BD
 - i. Candidato
 - 1. Id candidato
 - 2. Nombre
 - 3. correo
 - 4. años de experiencia
 - ii. Vacante
 - 1. id vacanet
 - 2. titulo de cargo
 - 3. años minimos de experiencia
 - 4. estado de vacante
 - iii. Postulacion
 - 1. id postulacion
 - 2. id candidato
 - 3. id vacante
 - 4. carta de presentacion
 - 5. fuente de postulacion

6. puntaje
7. prioridad
8. estado
9. fecha de creacion
10. fecha de ultima actualizacion de estado

e. Requerimientos de diseño:

- i. Diseñar las tablas
- ii. Tipos de datos
- iii. primary keys
- iv. foreign keys
- v. restricciones
- vi. demas elementos necesarios
- vii. explicar relaciones entre candidatos - vacantes - postulaciones
- viii. No se exige implementar endpoints para crear candidatos y vacantes
- ix. seed de al menos 3 candidatos y 2 vacantes (1 en estado CLOSED)

f. Valores Permitidos:

- i. Fuentes de postulacion
 1. REFERRAL
 2. INTERNAL
 3. JOB_BOARD
 4. OTHER
- ii. Estados de postulacion
 1. RECIEVED
 2. IN_REVIEW

3. REJECTED

4. HIRED

iii. Estado de Vacante

1. OPEN

2. CLOSED

3. Tomar en cuenta: se consideran postulaciones activas aquellas en estado RECIEVED o IN_REVIEW

4. Estados finales: REJECTED y HIRED

g. Calculo de Prioridad

i. Requerimiento: el calculo de prioridad se realiza en el backend, no es valor proporcionado por usuario.

ii. Condicion

1. +4

Los años de experiencia del candidato son iguales o superiores a los mínimos exigidos por la vacante

2. +3

La fuente es REFERRAL

3. +2

La fuente es INTERNAL

4. +2

La carta de presentación contiene alguna de las palabras "node", "sql" o "api"

5. +1

La carta de presentación tiene más de 500 caracteres

6. -2

El candidato tiene 3 o más postulaciones activas en otras vacantes

iii. Prioridad

1. TOP puntaje 7+
2. HIGH puntaje 5 - 6
3. MEDIUM puntaje 3 - 4
4. LOW puntaje 0 - 2

h. Funciones Obligatorias

i. Registrar una postulación

Implementar:

POST /applications

Ejemplo de cuerpo de solicitud:

```
{  
  "candidateId": 3,  
  "vacancyId": 1,  
  "source": "REFERRAL",  
  "coverLetter": "I have four years of experience building REST  
APIs with Node.js and SQL databases"  
}
```

La operación debe, como mínimo:

1. Validar los datos obligatorios.
2. Validar que la fuente de la postulación sea permitida.
3. Verificar que el candidato exista.
4. Verificar que la vacante exista y se encuentre en estado OPEN.
5. Aplicar la regla de duplicidad descrita en la sección 8.
6. Consultar la información necesaria para aplicar las reglas de prioridad.
7. Calcular el puntaje y determinar la prioridad.
8. Registrar la postulación en la base de datos.
9. Asignar RECEIVED como estado inicial.

10. Retornar una respuesta HTTP coherente con el resultado de la operación.

ii. Consultar postulaciones

Implementar:

GET /applications

La respuesta debe incluir la información de la postulación y, como mínimo, el nombre y correo electrónico del candidato y el título de la vacante.

Los resultados deben retornarse ordenados por puntaje de mayor a menor y, en caso de empate, por fecha de creación (la más antigua primero).

También debe permitir filtrar por estado, por vacante o por ambos criterios combinados:

GET /applications?status=IN_REVIEW

GET /applications?status=RECEIVED&vacancyId=1

iii. Cambiar el estado de una postulación

Implementar:

PUT /applications/:id/status

Ejemplo de cuerpo de solicitud:

```
{  
  "status": "IN_REVIEW"  
}
```

La operación debe validar:

- Que la postulación exista.
- Que el nuevo estado corresponda a uno de los estados permitidos.
- Que una postulación en estado final (REJECTED o HIRED) no pueda cambiar de estado.
- Que se actualice la fecha de última actualización de estado.
- Que los errores sean respondidos mediante códigos HTTP apropiados.

i. Regla de limitacion

- i. Los candidatos no pueden postularse a las vacantes que ya tienen postulado. Solamente pueden volver a postular cuando

hayan transcurrido minimop 30 dias desde la fecha de rechazo.

j. Implementacion de Pruebas

- i. implementar 3 priebas automatizadas relacionadas con la logica de calculo de puntaje y prioridad.

k. Fundamentos de IA

- i. Agregar un archivo de RESPUESTAS.md

- **Output (El "Cómo entregarlo"):**

Necesito que como primer paso escribamos el BRIEF del proyecto para comenzar a desarrollar. Toma en cuenta estos requerimientos finales para el proyecto. Entregables=

1. Código fuente de la aplicación.
2. package.json y archivo de bloqueo de dependencias generado por el gestor utilizado.
3. database.sql con las instrucciones necesarias para crear las tablas y estructuras de base de datos requeridas, junto con los datos de prueba de candidatos y vacantes.
4. .env.example con las variables de entorno necesarias, sin contraseñas o credenciales reales.
5. RESPUESTAS.md con las respuestas de la sección de inteligencia artificial.
6. README.md con instrucciones para instalar dependencias, configurar la base de datos, ejecutar la aplicación y probar los endpoints
7. Las pruebas automatizadas implementadas.
8. No incluya la carpeta node_modules en la entrega
9. Se valorará el uso adecuado de variables de entorno y la ausencia de credenciales sensibles en el código.

10. Se valorará la separación de responsabilidades, legibilidad, nombres adecuados y manejo consistente de errores.
11. Las decisiones técnicas que considere relevantes pueden explicarse brevemente en [README.md](#).

▼ BRIEF creado por Claude

Brief — API REST de Postulaciones a Vacantes

@September 30, 2026 · @Dan

Resumen ejecutivo

Construiremos una API REST en Node.js y PostgreSQL que registra postulaciones, calcula en el backend un puntaje (de -2 a 10) y una prioridad (TOP, HIGH, MEDIUM, LOW), y las entrega ordenadas para que el equipo de selección revise primero a los candidatos más afines.

El problema. La empresa recibe postulaciones sin orden útil. Los reclutadores pierden tiempo revisando perfiles poco afines antes que los fuertes. La API resuelve esto con reglas de negocio objetivas y aplicadas siempre igual.

Quién la usa.

- **Candidatos:** se postulan a una vacante abierta (`POST /applications`), normalmente a través de un frontend que no es parte de este proyecto.
- **Equipo de selección:** consulta postulaciones ordenadas y filtradas (`GET /applications`) y cambia su estado (`PUT /applications/:id/status`).

Rol asumido: Backend Engineer (Node.js) con responsabilidades de DBA. El peso del proyecto está en la lógica de negocio, el diseño relacional y el manejo de errores. Un *API Platform Integration Specialist* se enfoca en integrar plataformas externas, gateways o terceros, y aquí no hay integraciones. Un *Query Optimization Specialist* es demasiado estrecho para 3 tablas y pocos

datos. Sí tomaremos del DBA lo que pesa: restricciones, índices justificados por consultas reales y transacciones para evitar condiciones de carrera.

Cómo respondemos a cada criterio de evaluación.

Criterio	Cómo lo demostramos
Lógica aplicada	Cálculo de puntaje como función pura, probada, con desglose de reglas aplicadas
Calidad de decisiones técnicas	Restricciones en BD además del backend, transacciones, índices justificados, pocas dependencias
Organización del proyecto	Capas separadas: rutas, controladores, servicios, repositorios y dominio
Manejo de errores	Clase de error única, formato de respuesta JSON consistente, códigos HTTP correctos
Claridad de la solución	README con decisiones técnicas, ejemplos de uso y comentarios en el código

Alcance

El alcance son 3 endpoints sobre postulaciones, la base de datos con sus datos de prueba, 3 o más pruebas automatizadas y la documentación.

Incluye

- Esquema relacional de `candidates`, `vacancies` y `applications` con restricciones, índices y seed.
- `POST /applications`, `GET /applications` (con filtros) y `PUT /applications/:id/status`.
- Cálculo de puntaje y prioridad en el backend, con regla de duplicidad de 30 días.
- Manejo centralizado de errores y validaciones.
- Pruebas automatizadas del cálculo de puntaje y prioridad.
- `README.md`, `RESPUESTAS.md` y `.env.example`.

Fuera de alcance (se menciona en el README como posible extensión)

- Endpoints para crear o editar candidatos y vacantes (el enunciado lo excluye).

- Autenticación y roles (reclutador vs. candidato).
- Paginación del listado, frontend y despliegue.
- Historial de cambios de estado en una tabla aparte.

Stack técnico

Usaremos Node.js 22 LTS, Express 5 y PostgreSQL 16 con SQL escrito a mano, con solo 3 dependencias de producción y ninguna de desarrollo.

Componente	Elección	Por qué
Runtime	Node.js 22 LTS	Versión estable con soporte largo; trae <code>node:test</code> y <code>--watch</code> integrados
Framework HTTP	Express 5	Estándar conocido; la v5 pasa los errores de funciones <code>async</code> al manejador de errores sin librerías extra
Base de datos	PostgreSQL 16	Índices únicos parciales, <code>CHECK</code> , <code>TIMESTAMPTZ</code> y <code>RETURNING</code> : permiten proteger reglas de negocio en la propia BD
Acceso a datos	<code>pg</code> (node-postgres), sin ORM	Muestra dominio de SQL, consultas parametrizadas y transacciones explícitas
Configuración	<code>dotenv</code>	Carga <code>.env</code> de forma portátil; ninguna credencial queda en el código
Validación	Validadores propios	Pocas reglas; escribirlas deja la lógica visible y evita otra dependencia
Pruebas	<code>node:test</code> + <code>node:assert</code>	Incluido en Node; cero dependencias de desarrollo

¿Por qué PostgreSQL y no MySQL? Ambas cumplen "relacional basada en SQL". PostgreSQL permite un índice único **parcial** que impide, a nivel de BD, dos postulaciones activas del mismo candidato a la misma vacante. En MySQL esa garantía requiere triggers o columnas generadas. Si la empresa exige MySQL, el diseño se adapta con cambios menores.

Consultas siempre parametrizadas (`$1`, `$2`...): nunca se concatenan valores del usuario en SQL, lo que evita inyección SQL.

Modelo de datos

`applications` es la tabla central: une candidatos con vacantes (relación muchos a muchos) y guarda los datos propios de cada postulación.

[embed: node/37f64646-44df]

Un candidato y una vacante pueden tener muchas postulaciones; cada postulación apunta a exactamente un candidato y una vacante.

Relaciones

- Un candidato tiene 0 o muchas postulaciones; cada postulación pertenece a 1 candidato.
- Una vacante recibe 0 o muchas postulaciones; cada postulación pertenece a 1 vacante.
- Por eso `candidates` ↔ `vacancies` es N:M, resuelta por `applications`, que además tiene atributos propios (puntaje, estado, fechas).
- La PK de `applications` es un `id` propio y **no** el par (candidato, vacante): un candidato puede volver a postularse a la misma vacante 30 días después de un rechazo, así que el par puede repetirse en el historial.

Convenciones: nombres en inglés y `snake_case` en la BD; `camelCase` en el JSON de la API (el repositorio hace la traducción). PK con `GENERATED ALWAYS AS IDENTITY` (el estándar SQL moderno frente a `SERIAL`). Fechas en `TIMESTAMPTZ` (UTC). Valores permitidos con `VARCHAR` + `CHECK` en lugar de `ENUM` nativo: se modifican con un `ALTER` simple.

candidates

Columna	Tipo	Restricciones
id	INTEGER	PK, identity
full_name	VARCHAR(150)	NOT NULL, no vacío (<code>CHECK (length(trim(full_name)) > 0)</code>)
email	VARCHAR(255)	NOT NULL, único sin distinguir mayúsculas (índice único sobre <code>LOWER(email)</code>)
years_of_experience	SMALLINT	NOT NULL, <code>CHECK (years_of_experience BETWEEN 0 AND 60)</code>

Columna	Tipo	Restricciones
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT <code>now()</code>

vacancies

Columna	Tipo	Restricciones
id	INTEGER	PK, identity
title	VARCHAR(150)	NOT NULL, no vacío
min_years_experience	SMALLINT	NOT NULL, <code>CHECK (min_years_experience BETWEEN 0 AND 60)</code>
status	VARCHAR(10)	NOT NULL, DEFAULT <code>'OPEN'</code> , <code>CHECK (status IN ('OPEN', 'CLOSED'))</code>
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT <code>now()</code>

applications

Columna	Tipo	Restricciones
id	INTEGER	PK, identity
candidate_id	INTEGER	NOT NULL, FK → <code>candidates(id)</code> <code>ON DELETE RESTRICT</code>
vacancy_id	INTEGER	NOT NULL, FK → <code>vacancies(id)</code> <code>ON DELETE RESTRICT</code>
cover_letter	TEXT	NULL permitido, <code>CHECK (char_length(cover_letter) <= 5000)</code>
source	VARCHAR(20)	NOT NULL, <code>CHECK (source IN ('REFERRAL', 'INTERNAL', 'JOB_BOARD', 'OTHER'))</code>
score	SMALLINT	NOT NULL, <code>CHECK (score BETWEEN -2 AND 10)</code>
priority	VARCHAR(10)	NOT NULL, <code>CHECK (priority IN ('TOP', 'HIGH', 'MEDIUM', 'LOW'))</code>
status	VARCHAR(20)	NOT NULL, DEFAULT <code>'RECEIVED'</code> , <code>CHECK (status IN ('RECEIVED', 'IN_REVIEW', 'REJECTED', 'HIRED'))</code>
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT <code>now()</code>
status_updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT <code>now()</code>

Restricciones adicionales en `applications`

- **Coherencia puntaje–prioridad:** un `CHECK` que exige, por ejemplo, `score >= 7` ⇔ `priority = 'TOP'`. El backend calcula ambos; la BD rechaza cualquier combinación incoherente. Es una red de seguridad, no la fuente del cálculo.
- **Fechas coherentes:** `CHECK (status_updated_at >= created_at)`.
- `ON DELETE RESTRICT` en ambas FK: no se puede borrar un candidato o vacante con postulaciones, para no perder historial.

Índices (cada uno responde a una consulta concreta)

Índice	Columnas	Consulta que acelera
<code>uq_applications_active_pair</code>	<code>(candidate_id, vacancy_id)</code> único parcial <code>WHERE status IN ('RECEIVED', 'IN_REVIEW')</code>	Garantiza en la BD que no existan dos postulaciones activas iguales
<code>idx_applications_candidate_vacancy</code>	<code>(candidate_id, vacancy_id, created_at DESC)</code>	Regla de duplicidad y conteo de postulaciones activas del candidato
<code>idx_applications_vacancy_status</code>	<code>(vacancy_id, status)</code>	Filtros de <code>GET /applications</code> por vacante y estado
<code>idx_applications_ranking</code>	<code>(score DESC, created_at ASC)</code>	Orden del listado

PostgreSQL no crea índices automáticos para las FK; los dos primeros índices ya cubren `candidate_id` y el tercero cubre `vacancy_id`. Con pocos datos el planificador puede ignorarlos, pero dejan el diseño listo para crecer.

Dato clave de diseño: no hace falta una columna `rejected_at`. Como un estado final no puede cambiar, en una postulación `REJECTED` el valor de `status_updated_at` es la fecha de rechazo.

Reglas de negocio

El puntaje va de -2 a 10 y se calcula una sola vez, al crear la postulación, con una función pura que no toca la BD ni HTTP.

Cálculo de puntaje

Regla	Puntos	Cómo se implementa
Experiencia del candidato $\geq$ mínimo de la vacante	+4	<code>candidate.years_of_experience >= vacancy.min_years_experience</code>
Fuente REFERRAL	+3	Excluyente con INTERNAL (una postulación tiene una sola fuente)
Fuente INTERNAL	+2	
Carta contiene "node", "sql" o "api"	+2	Una sola vez aunque aparezcan varias; sin distinguir mayúsculas
Carta con más de 500 caracteres	+1	Estrictamente > 500, tras <code>trim()</code> , contando caracteres Unicode reales
3 o más postulaciones activas en otras vacantes	-2	<code>COUNT(*)</code> con <code>status IN ('RECEIVED', 'IN_REVIEW')</code> AND <code>vacancy_id <> :vacancyId</code>

Máximo posible: $4 + 3 + 2 + 1 = 10$. Mínimo posible: **-2**.

Detección de palabras clave. Una búsqueda simple de texto da falsos positivos: "capital" y "rapid" contienen "api". Proponemos la expresión

`/\bnode|sql|bapi/i`:

- Cuentan: "Node.js", "nodes", "APIs", "MySQL", "PostgreSQL", "NoSQL".
- No cuentan: "capital", "rapid", "therapist".

La decisión se documenta en el README y se cubre con una prueba.

Prioridad

Prioridad	Puntaje
TOP	7 o más
HIGH	5 a 6

Prioridad	Puntaje
MEDIUM	3 a 4
LOW	2 o menos (incluye 0, -1 y -2)

Guardamos el puntaje real (puede ser negativo) para que el orden del listado sea preciso, y la prioridad LOW cubre todo lo que está por debajo de 3.

Puntaje como "fotografía". El puntaje refleja la situación al momento de postular. Si luego cambian otras postulaciones del candidato, no se recalcula. Así el ranking es estable y auditable.

Regla de duplicidad (30 días)

Se busca la postulación **más reciente** del mismo candidato a la misma vacante:

1. No existe → se permite.
2. Está en `RECEIVED` o `IN_REVIEW` → **409**, ya hay una postulación activa.
3. Está en `HIRED` → **409**, el candidato ya fue contratado para esa vacante (supuesto; ver preguntas).
4. Está en `REJECTED` y `status_updated_at` es posterior a `now() - interval '30 days'` → **409**, con la fecha desde la que podrá volver a postularse (`canReapplyAt`).
5. Está en `REJECTED` hace 30 días o más → se permite.

La comparación se hace en SQL con el reloj de la BD, para no depender de diferencias de hora entre servidor y BD.

Condiciones de carrera. Si llegan dos postulaciones iguales al mismo tiempo, ambas podrían pasar la validación. Lo evitamos así:

- Todo el registro corre en **una transacción**.
- Se bloquea la fila del candidato con `SELECT ... FOR UPDATE`. Las postulaciones del mismo candidato se procesan una tras otra, y el conteo de activas es exacto.
- El índice único parcial es la última defensa: si igual se viola (código `23505`), se responde 409.

Estados y transiciones

[embed: node/a388ba86-765d]

Un cambio de estado solo parte de RECEIVED o IN_REVIEW; desde REJECTED o HIRED la API responde 409 y no toca la fila.

- Estados activos: `RECEIVED`, `IN_REVIEW`. Estados finales: `REJECTED`, `HIRED`.
- Desde un estado activo se puede pasar a cualquier otro estado permitido y distinto al actual.
- Desde un estado final no se puede cambiar: **409**.
- Cada cambio actualiza `status_updated_at = now()`.
- La actualización es atómica: `UPDATE ... WHERE id = $1 AND status IN ('RECEIVED', 'IN_REVIEW') RETURNING *`. Si no afecta filas, una consulta aparte distingue entre 404 (no existe) y 409 (ya es final).

Ejemplo con el caso del enunciado

Candidato 3 (4 años de experiencia) se postula a la vacante 1 (mínimo 3 años) con fuente `REFERRAL` y la carta "I have four years of experience building REST APIs with Node.js and SQL databases":

- Experiencia $4 \geq 3 \rightarrow +4$
- `REFERRAL` $\rightarrow +3$
- Contiene "APIs", "Node" y "SQL" $\rightarrow +2$
- La carta tiene unos 80 caracteres $\rightarrow +0$
- Sin otras postulaciones activas $\rightarrow +0$

Puntaje **9** $\rightarrow$ prioridad **TOP**. El seed se diseñará para que este ejemplo funcione tal cual.

Especificación de endpoints

Los 3 endpoints responden JSON; las validaciones van de lo barato a lo costoso: primero el formato, luego la existencia en BD y por último las reglas de negocio.

POST /applications — registrar postulación

```
{
  "candidateId": 3,
  "vacancyId": 1,
  "source": "REFERRAL",
  "coverLetter": "I have four years of experience building
REST APIs with Node.js and SQL databases"
}
```

Paso	Validación	Si falla
1	El cuerpo es JSON válido	400 <code>INVALID_JSON</code>
2	<code>candidateId</code> y <code>vacancyId</code> presentes, enteros positivos	400 <code>VALIDATION_ERROR</code>
3	<code>source</code> presente y en <code>REFERRAL</code> , <code>INTERNAL</code> , <code>JOB_BOARD</code> , <code>OTHER</code>	400 <code>VALIDATION_ERROR</code>
4	<code>coverLetter</code> opcional; si viene, texto de hasta 5000 caracteres	400 <code>VALIDATION_ERROR</code>
5	Se ignoran <code>score</code> , <code>priority</code> y <code>status</code> si el cliente los envía	—
6	El candidato existe	404 <code>CANDIDATE_NOT_FOUND</code>
7	La vacante existe	404 <code>VACANCY_NOT_FOUND</code>
8	La vacante está <code>OPEN</code>	409 <code>VACANCY_NOT_OPEN</code>
9	Regla de duplicidad de 30 días	409 <code>DUPLICATE_APPLICATION</code>
10	Cálculo de puntaje, prioridad e inserción con estado <code>RECEIVED</code>	201 Created

Los errores de formato (pasos 2 a 4) se juntan en una sola respuesta, para que el cliente corrija todo de una vez. Los pasos 6 a 10 ocurren dentro de la transacción.

Respuesta **201** con cabecera `Location: /applications/{id}`. Incluye el desglose del puntaje para que el reclutador entienda la prioridad (no se guarda en BD):

```
{
  "data": {
    "id": 12,
    "candidateId": 3,
    "vacancyId": 1,
    "source": "REFERRAL",
    "coverLetter": "I have four years...",
    "score": 9,
    "priority": "TOP",
    "status": "RECEIVED",
    "createdAt": "2026-10-01T15:20:00.000Z",
    "statusUpdatedAt": "2026-10-01T15:20:00.000Z",
    "scoreBreakdown": [
      { "rule": "MEETS_MIN_EXPERIENCE", "points": 4 },
      { "rule": "SOURCE_REFERRAL", "points": 3 },
      { "rule": "COVER_LETTER_KEYWORDS", "points": 2 }
    ]
  }
}
```

GET /applications — consultar postulaciones

Parámetro	Validación	Si falla
<code>status</code> (opcional)	Uno de los 4 estados permitidos	400 <code>VALIDATION_ERROR</code>
<code>vacancyId</code> (opcional)	Entero positivo	400 <code>VALIDATION_ERROR</code>

- Los filtros se combinan con `AND`. Sin filtros, se devuelven todas.
- Orden: `ORDER BY score DESC, created_at ASC, id ASC`. El `id` final garantiza un orden estable si dos postulaciones tienen el mismo puntaje y fecha.
- `JOIN` con `candidates` y `vacancies` en una sola consulta (sin consultas extra por fila).
- Si el filtro no encuentra nada (incluso una vacante que no existe), responde **200** con lista vacía: filtrar no es buscar un recurso puntual.

- La consulta se arma con condiciones fijas y parámetros, nunca concatenando texto del usuario.

```
{
  "count": 1,
  "data": [
    {
      "id": 12,
      "status": "RECEIVED",
      "source": "REFERRAL",
      "score": 9,
      "priority": "TOP",
      "coverLetter": "I have four years...",
      "createdAt": "2026-10-01T15:20:00.000Z",
      "statusUpdatedAt": "2026-10-01T15:20:00.000Z",
      "candidate": { "id": 3, "fullName": "...", "email": "..."},
      "vacancy": { "id": 1, "title": "Backend Developer Node.js" }
    }
  ]
}
```

PUT /applications/:id/status — cambiar estado

```
{ "status": "IN_REVIEW" }
```

Paso	Validación	Si falla
1	<code>:id</code> es entero positivo	400 <code>VALIDATION_ERROR</code>
2	<code>status</code> presente y permitido	400 <code>VALIDATION_ERROR</code>
3	La postulación existe	404 <code>APPLICATION_NOT_FOUND</code>
4	El estado actual no es final	409 <code>APPLICATION_IN_FINAL_STATE</code>

Paso	Validación	Si falla
5	El nuevo estado es distinto al actual	409 <code>STATUS_UNCHANGED</code>
6	Se actualiza <code>status</code> y <code>status_updated_at</code>	200 con la postulación actualizada

Criterio de códigos HTTP

- **400**: la petición está mal formada o tiene datos inválidos. El cliente debe corregirla.
- **404**: el recurso referido no existe.
- **409**: la petición es válida, pero choca con el estado actual de los datos (vacante cerrada, duplicado, estado final). Se usa 409 de forma consistente para todas las reglas de negocio.
- **201**: recurso creado. **200**: consulta o actualización correcta.
- **500**: error inesperado; nunca expone detalles internos.

Arquitectura y estructura de carpetas

Usaremos capas: cada petición pasa por ruta → controlador → servicio → repositorio, y el cálculo de puntaje vive aparte, en el dominio, sin depender de Express ni de la BD.

Capa	Responsabilidad	No debe
Rutas	Asociar método + URL a un controlador	Contener lógica
Validadores	Revisar formato de body, params y query	Consultar la BD
Controladores	Leer la petición, llamar al servicio, armar la respuesta HTTP	Conocer SQL ni reglas de negocio
Servicios	Orquestar el caso de uso: existencia, reglas, transacción	Conocer <code>req</code> / <code>res</code>
Repositorios	Ejecutar SQL parametrizado y traducir <code>snake_case</code> ↔ <code>camelCase</code>	Decidir reglas de negocio
Dominio	Constantes, cálculo de puntaje y prioridad (funciones puras)	Tener efectos secundarios

```

applications-api/
├── src/
│   ├── app.js                # Crea la app Express, middlewares y rutas
│   └── server.js             # Arranca el servidor (separado para poder proba
r app.js)
│   ├── config/
│   │   └── env.js            # Lee y valida variables de entorno al iniciar
│   ├── db/
│   │   └── pool.js           # Pool de pg + helper withTransaction()
│   ├── domain/
│   │   └── constants.js      # SOURCES, STATUSES, ACTIVE_STATUSES, FINAL_STAT
USES...
│   │   └── scoring.js        # calculateScore() y getPriority()
│   ├── validators/
│   │   └── applications.validator.js
│   ├── routes/
│   │   └── applications.routes.js
│   ├── controllers/
│   │   └── applications.controller.js
│   ├── services/
│   │   └── applications.service.js
│   ├── repositories/
│   │   ├── applications.repository.js
│   │   ├── candidates.repository.js
│   │   └── vacancies.repository.js
│   └── errors/
│       ├── AppError.js       # Error con statusCode, code y details
│       └── errorHandler.js   # Middleware 404 y manejador central
├── tests/
│   └── scoring.test.js
├── database.sql
├── requests.http              # Peticiones de ejemplo (REST Client / Postman)
├── .env.example
├── .gitignore                 # node_modules, .env
├── package.json
├── package-lock.json
├── README.md
└── RESPUESTAS.md

```

Scripts de `package.json`: `npm start` (producción), `npm run dev` (con `node --watch`), `npm test` (`node --test`).

Variables de entorno (`.env.example`, sin valores reales): `PORT`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`. `config/env.js` detiene el arranque con un mensaje claro si falta alguna obligatoria.

Manejo de errores

Todo error sale por un único middleware y con el mismo formato JSON, con un `code` estable que el cliente puede usar sin leer el mensaje.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "La solicitud contiene datos inválidos",
    "details": [
      { "field": "source", "message": "Debe ser uno de: REFERRAL, INTERNAL, JOB_BOARD, OTHER" }
    ]
  }
}
```

Cómo funciona

1. Validadores y servicios lanzan `AppError(statusCode, code, message, details?)`.
2. Express 5 envía al manejador los errores de funciones `async` sin `try/catch` repetidos.
3. El manejador traduce también errores que no son `AppError`:

Origen	Traducción
JSON mal formado (<code>SyntaxError</code> de <code>express.json()</code>)	400 <code>INVALID_JSON</code>
PostgreSQL <code>23505</code> (único violado)	409 <code>DUPLICATE_APPLICATION</code>
PostgreSQL <code>23503</code> (FK violada)	404 del recurso relacionado
PostgreSQL <code>23514</code> (<code>CHECK</code> violado)	400 <code>VALIDATION_ERROR</code>
Ruta inexistente	404 <code>ROUTE_NOT_FOUND</code>
Cualquier otro	500 <code>INTERNAL_ERROR</code> , mensaje genérico; el detalle solo va al log del servidor

1. Si falla algo dentro de una transacción, se hace `ROLLBACK` y la conexión vuelve al pool en un bloque `finally` , para no dejar datos a medias ni conexiones colgadas.

2. En `409 DUPLICATE_APPLICATION` por rechazo reciente, `details` incluye `canReapplyAt`.

Pruebas automatizadas

El mínimo son 3 pruebas; proponemos 6, todas sobre `domain/scoring.js`, que al ser funciones puras se prueban sin BD ni servidor.

#	Prueba	Qué demuestra
1	Caso del enunciado: <code>REFERRAL</code> , experiencia suficiente, carta con palabras clave → 9, TOP	Las reglas se suman bien en el caso real
2	Caso máximo: todas las bonificaciones → 10, TOP	Suma completa y tope
3	Penalización: 3 activas en otras vacantes resta 2; con 2 activas no resta	El límite exacto de la regla -2
4	Límites de prioridad: 7 → TOP, 6 y 5 → HIGH, 4 y 3 → MEDIUM, 2, 0 y -2 → LOW	Cada frontera de la tabla de prioridad
5	Carta de 500 caracteres no suma; de 501 sí	"Más de 500" es estricto
6	"NODE" suma (mayúsculas); "capital" y "rapid" no suman	Palabras clave sin falsos positivos

`calculateScore()` recibe un objeto simple y devuelve `{ score, breakdown }`:

```
calculateScore({
  candidateYears: 4,
  minYears: 3,
  source: 'REFERRAL',
  coverLetter: '...',
  otherActiveApplications: 0
}); // → { score: 9, breakdown: [...] }
```

Pruebas de integración de los endpoints (con una BD de prueba) quedan como extensión opcional si hay tiempo.

Entregables y checklist

Son 7 entregables más 4 criterios de calidad; esta lista sirve para revisar antes de entregar.

Entregables

- Código fuente en `src/`
- `package.json` y `package-lock.json` (generado por npm)
- `database.sql` : crea tablas, restricciones e índices, más el seed de candidatos y vacantes; se puede ejecutar varias veces sin error (`DROP TABLE IF EXISTS` al inicio)
- `.env.example` sin credenciales reales
- `RESPUESTAS.md` con las respuestas de la sección de IA
- `README.md` : instalación, configuración de BD, ejecución, pruebas de endpoints con `curl` y decisiones técnicas
- Pruebas automatizadas en `tests/` , que pasan con `npm test`

Criterios de calidad

- `node_modules/` y `.env` fuera de la entrega (`.gitignore`)
- Ninguna credencial en el código; todo desde variables de entorno
- Capas separadas, nombres claros y comentarios donde la lógica no es obvia
- Todos los errores con el mismo formato JSON y el código HTTP correcto

Diseño del seed. El enunciado pide mínimo 3 candidatos y 2 vacantes (1 `CLOSED`). Con solo 2 vacantes **no se puede probar la regla -2**: un candidato necesita 3 postulaciones activas en otras vacantes más la vacante nueva, o sea 4 vacantes abiertas. Proponemos:

Vacante	Título	Mínimo de años	Estado
1	Backend Developer Node.js	3	OPEN
2	Data Analyst SQL	2	OPEN
3	Frontend Developer	1	OPEN

Vacante	Título	Mínimo de años	Estado
4	QA Engineer	2	OPEN
5	DevOps Engineer	5	CLOSED

Candidato	Nombre	Años de experiencia	Para probar
1	Ana López	6	Cumple todos los mínimos
2	Carlos Méndez	1	No cumple el mínimo de la vacante 1
3	María Pérez	4	El ejemplo del enunciado → 9, TOP
4	José Ramírez	2	Penalización -2 tras 3 postulaciones activas

Los correos usarán el dominio reservado `example.com`, que no pertenece a nadie.

Supuestos y preguntas abiertas

Solo la pregunta 1 bloquea un entregable; para las demás aplicaremos la propuesta indicada si no hay otra indicación, y la documentaremos en el README.

Corrección detectada: en tu entendimiento aparece `RECIEVED`; el enunciado usa `RECEIVED`, que es la forma correcta en inglés. Usaremos `RECEIVED` en todo el sistema.

#	Pregunta	Propuesta por defecto
1	¿Cuáles son las preguntas de la sección "Fundamentos de IA" para <code>RESPUESTAS.md</code> ? No venían en el entendimiento.	Bloqueante: necesitamos el texto de las preguntas
2	¿El motor de BD es libre o piden MySQL?	PostgreSQL 16
3	¿ <code>coverLetter</code> es obligatoria?	Opcional; sin carta, esas dos reglas suman 0

#	Pregunta	Propuesta por defecto
4	¿Un candidato <code>HIRED</code> puede volver a postularse a la misma vacante?	No, se responde 409
5	¿Cambiar al mismo estado (p. ej. <code>IN_REVIEW</code> → <code>IN_REVIEW</code>) es error?	Sí, 409 <code>STATUS_UNCHANGED</code> , para que <code>status_updated_at</code> refleje cambios reales
6	¿Hay transiciones prohibidas entre estados activos (<code>IN_REVIEW</code> → <code>RECEIVED</code>) o saltos (<code>RECEIVED</code> → <code>HIRED</code>)?	Todas permitidas; el enunciado solo prohíbe salir de un estado final
7	¿"MySQL" o "APIs" cuentan como palabra clave?	Sí, con la expresión <code>`\bnode</code>
8	¿Un puntaje negativo se guarda tal cual o se lleva a 0?	Se guarda tal cual (−2 a 10) y cae en LOW
9	¿Los años de experiencia pueden ser decimales (2.5)?	No, enteros
10	¿Incluimos en el seed postulaciones de ejemplo (una rechazada hace 45 días y otra hace 10) para probar la regla de 30 días?	Sí, con puntaje calculado con las mismas reglas

Plan de desarrollo por fases

Son 7 fases en orden; cada una termina con algo que se puede ejecutar o probar antes de pasar a la siguiente.

1. **Base de datos:** escribir `database.sql` (tablas, restricciones, índices, seed) y ejecutarlo en PostgreSQL local. Verificar que las restricciones rechazan datos inválidos.
2. **Esqueleto del proyecto:** `npm init`, dependencias, `.gitignore`, `.env.example`, `config/env.js`, `db/pool.js`, `app.js`, `server.js`, manejador de errores y ruta 404.
3. **Dominio y pruebas:** `constants.js`, `scoring.js` y las 6 pruebas. Se escriben juntos: primero la prueba, luego la función.

4. **POST /applications:** validador, repositorios, servicio con transacción y bloqueo, controlador y ruta.
5. **GET /applications:** filtros, `JOIN` y orden.
6. **PUT /applications/:id/status:** `UPDATE` atómico y distinción 404 / 409.
7. **Documentación y cierre:** `requests.http`, `README.md`, `RESPUESTAS.md`, prueba manual de todos los casos de error y revisión con el checklist.

Siguiente paso: responder las preguntas abiertas (sobre todo la 1) y arrancar con la fase 1.

▼ Respuestas del RESPUESTAS.md

Pregunta 1. La empresa desea utilizar un modelo de inteligencia artificial para identificar automáticamente las habilidades técnicas del candidato a partir de su carta de presentación y compararlas con las requeridas por la vacante. Explique cómo integraría esta capacidad a la solución desarrollada.

Como primer punto creo que si se podría integrar la IA al sistema actual. Se podría poner una automatización del proceso del análisis de datos para poder dar un puntaje y realizar un filtro rápido antes de llegar al equipo evaluador. Creo que podemos integrar un OCR para evaluar la data proporcionada por el candidato, evaluar con las palabras clave dentro relacionadas a la vacante propuestas por el equipo evaluador. Luego se compara dentro del sistema y se le da un puntaje definitivo. Las limitaciones del sistema que son doble postulación, más de una postulación, etc podrían servir como filtro definitivo y eliminar. Aunque si recomiendo que alguien del equipo sea el encargado a evaluar todo el sistema. Al final el objetivo principal es encontrar buenos candidatos, no que el sistema elimine varios solo porque si.

Pregunta 2. Suponga que el modelo de IA devuelve ocasionalmente una respuesta con formato inválido o habilidades que no existen en el catálogo de la empresa. Explique

cómo debería
manejar esta situación el backend.

El sistema puede dar respuestas que no existen o que no están evaluadas por el equipo. Cada aplicante tiene sus propias maneras de implementar o venderse con sus habilidades. Cambios de lenguaje, de keywords, de idioma e incluso un sistema de evaluación que no ha sido actualizado pueden llegar a afectar. El backend debería tomar en cuenta este tipo de respuestas y dar una revisión humana. Se detecta los errores o respuestas inválidas, tomar en cuenta las habilidades que no existen y categorizarlo para revisión humana.

Pregunta 3. ¿Considera apropiado reemplazar completamente las reglas determinísticas utilizadas para calcular la prioridad por una decisión realizada por un modelo de IA, teniendo en cuenta que la decisión afecta directamente a personas? Justifique técnicamente su respuesta.

No, creo que el sistema puede ver mejores resultados al momento de utilizar estos dos sistemas IA + Humanos. Creo que cualquier error dentro del sistema no debe de afectar a los vacantes ni cortarlos solamente porque el diseñador no supo cómo referir o integrarlos. Creo que definitivamente debemos de dejar una revisión humana para no afectar totalmente a los vacantes.

REPO: Evaluacion_Daniel_Borja